



Data Analytics and Visualization

Piotr Jastrzębski

2026-03-21

Table of contents

1	Data Analysis and Visualization	5
I	Introduction	6
2	A bit of theory...	7
2.1	Rational thinking test	7
2.2	Data analysis	7
2.3	Data visualization	7
2.4	Data analysis - basic concepts	8
2.4.1	Modern meanings of the word “statistics”:	8
2.4.2	“Massiveness”	8
2.4.3	Division of statistics	9
2.4.4	Population	9
2.4.5	Statistical unit	9
2.4.6	Statistical features	9
2.4.7	Scales	11
2.5	Types of statistical studies	12
2.6	Stages of a statistical study	13
2.7	Analysis of existing data	13
2.8	Data analysis process	14
2.8.1	Defining requirements	14
2.8.2	Data collection	14
2.8.3	Data processing	14
2.8.4	Proper data analysis	15
2.8.5	Reporting and distribution of results	15
2.9	Where to get data?	15
2.10	“Tidy data” concept	15
2.10.1	“Tidy data” principles	16
2.10.2	Examples of untidy data	16
2.11	A few tips for good presentations	16
2.11.1	Lie factor	17
2.11.2	Lie factor	17
2.12	References	18

3	Configuration	19
II	NumPy	21
4	NumPy - Getting Started	22
4.1	Importing the NumPy library	22
5	List vs Array	24
6	Array Attributes	25
7	Data Types	27
8	Creating Arrays	28
9	Indexing, “slicing”	35
10	Modifying shape and size	41
11	Broadcasting	47
12	Universal functions (ufunc) and other mathematical functions	50
12.1	Basic arithmetic operations	50
12.2	Trigonometric and related functions	51
12.3	Exponential and logarithmic functions	51
12.4	Rounding functions and absolute values	52
12.5	Statistical and aggregate functions	53
13	String operations	54
13.1	Creating arrays with strings	54
13.2	Basic text modification functions	54
13.2.1	<code>numpy.strings.upper</code> and <code>numpy.strings.lower</code>	54
13.2.2	<code>numpy.strings.capitalize</code>	55
13.2.3	<code>numpy.strings.title</code>	55
13.3	Joining and splitting text	56
13.3.1	<code>numpy.strings.add</code>	56
13.3.2	<code>numpy.char.join</code>	56
13.3.3	<code>numpy.char.split</code>	56
13.4	Searching and replacing substrings	57
13.4.1	<code>numpy.strings.find</code> and <code>numpy.strings.rfind</code>	57
13.4.2	<code>numpy.strings.replace</code>	57
13.5	Removing unwanted characters	58
13.5.1	<code>numpy.strings.strip</code> , <code>numpy.strings.lstrip</code> and <code>numpy.strings.rstrip</code>	58

14 Linear Algebra	59
14.1 Dot Product	59
14.2 Matrix Multiplication	59
14.3 Matrix-Vector Multiplication	60
14.4 Solving Systems of Linear Equations	60
14.5 Matrix Determinant	61
14.6 Eigenvalues and Eigenvectors	61
14.7 Singular Value Decomposition (SVD)	62
14.8 Matrix/Vector Norm	62
14.9 Inverse Matrix	63
14.10The <code>numpy.inner</code> Function - Inner Product	64
14.11The <code>numpy.outer</code> Function - Outer Product	64
14.12The <code>numpy.matmul</code> Function - Matrix Multiplication	65
15 Advanced Filtering	66
15.1 The <code>nonzero()</code> Function	66
15.2 The <code>where()</code> Function	67
15.3 The <code>indices()</code> and <code>ix_()</code> Functions	67
15.3.1 <code>indices()</code>	67
15.3.2 <code>ix_()</code>	68
15.4 <code>ogrid</code> and Grid Operations	69
15.5 The <code>ravel_multi_index()</code> and <code>unravel_index()</code> Functions	70
15.6 Diagonal Indices	70
15.7 The <code>take()</code> Function	71
15.8 The <code>take_along_axis()</code> Function	71
15.9 The <code>choose()</code> Function	72
15.10The <code>compress()</code> Function	72
15.11The <code>diag()</code> and <code>diagonal()</code> Functions	73
15.12The <code>select()</code> Function	73
15.13The <code>place()</code> Function	74
15.14The <code>put()</code> Function	74
15.15The <code>put_along_axis()</code> Function	75
15.16The <code>putmask()</code> Function	75
15.17The <code>fill_diagonal()</code> Function	75
16 NumPy - Miscellaneous	77
16.1 Constants	77
16.2 <code>numpy.inf</code>	78
16.3 <code>numpy.nan</code>	79
16.4 <code>numpy.newaxis</code>	79
16.5 Statistics and Aggregation	80

1 Data Analysis and Visualization

The current version pertains to classes held in the 2025/26 academic year.

Some of the texts have been machine-translated into English. In case of any ambiguities, please get in touch via email.

Part I

Introduction

2 A bit of theory...

2.1 Rational thinking test

- If 5 machines produce 5 devices in 5 minutes, how long will it take 100 machines to make 100 devices?
- A patch of water lilies is growing on a pond. Every day the patch doubles in size. If it takes 48 days for the lilies to cover the entire pond, how many days does it take for them to cover half the pond?
- A baseball bat and a ball cost \$1.10 together. The bat costs one dollar more than the ball. How much does the ball cost?

2.2 Data analysis

Data analysis is the process of examining, cleaning, transforming, and modeling data in order to discover useful information, draw conclusions, and support decision-making. It is a multi-stage process that includes:

- Collecting data from various sources
- Cleaning data by removing errors, missing values, and inconsistencies
- Exploring data to understand its structure and characteristics
- Transforming data into the appropriate format
- Applying statistical methods and machine learning algorithms
- Interpreting results in the context of a specific business or scientific problem

Data analysis is used in nearly every field, from business and finance to social sciences, medicine, and scientific research. The goal of data analysis is to transform raw data into knowledge that can be used to make better decisions.

2.3 Data visualization

Data visualization is the graphical representation of information and data. It uses visual elements such as charts, maps, and dashboards to present relationships between data in a way that is easy to understand and interpret. Good data visualization:

- Presents complex information in an accessible and intuitive way
- Reveals patterns, trends, and outliers that may be difficult to notice in raw data
- Supports the data analysis process by enabling quick review of large datasets
- Facilitates communication of analysis results to various audiences, including non-technical individuals
- Helps tell stories contained in data (data storytelling)

The most popular types of data visualization include bar charts, line charts, pie charts, heatmaps, hierarchical trees, word clouds, and interactive dashboards. The choice of the appropriate visualization form depends on the type of data, the purpose of the presentation, and the target audience.

Data visualization is a key element of the data analysis process because it allows for quick drawing of conclusions and making decisions based on data. It is a bridge between complex data and human understanding.

2.4 Data analysis - basic concepts

2.4.1 Modern meanings of the word “statistics”:

- a set of numerical data showing the development of processes and phenomena, e.g. population statistics.
- all activities related to collecting and processing numerical data, e.g. statistics on a certain problem conducted by the Central Statistical Office.
- numerical characteristics, e.g. sample statistics such as arithmetic mean, standard deviation, etc.
- a scientific discipline - the science of methods for studying mass phenomena.

2.4.2 “Massiveness”

Mass phenomena/processes - a large number of units are subject to study. They are divided into:

- economic (e.g. production, consumption, services, advertising),
- social (e.g. traffic accidents, political views),
- demographic (e.g. births, aging, migrations).

2.4.3 Division of statistics

Statistics - a scientific discipline - division:

- descriptive statistics - deals with matters related to collecting, presenting, analyzing, and interpreting numerical data. Observation covers the entire population under study.
- mathematical statistics - generalization of the results of studying a part of the population (sample) to the entire population.

2.4.4 Population

Statistical population: a set of objects subject to statistical study. It consists of units that are similar to each other, logically related, but not identical. They share certain common features and certain properties that allow them to be differentiated.

- examples:
 - study of the height of Polish citizens - inhabitants of Poland
 - level of education in schools of the Warmian-Masurian Voivodeship - schools of the Warmian-Masurian Voivodeship.
- division:
 - general population - covers the entirety,
 - sample population (sample) - covers a part of the population.

2.4.5 Statistical unit

Statistical unit: each element of a statistical population.

- examples:
 - UWM students - a UWM student
 - inhabitants of Poland - every person living in Poland
 - machines produced in a factory - every machine

2.4.6 Statistical features

Statistical features

- properties characterizing statistical units in a given statistical population.
- they are divided into constant and variable.

Constant features

- properties that are common to all units of a given statistical population.
- division:
 - substantive - who or what is the subject of the statistical study,
 - temporal - when the study was conducted or what time period the study covers,
 - spatial - what territory (place or area) the study concerns.
- example: students of the Faculty of Mathematics and Computer Science (WMiI) at UWM in Olsztyn in the academic year 2017/2018:
 - substantive feature: possession of a student ID card,
 - temporal feature - students studying in the academic year 2017/2018,
 - spatial feature - location: WMiI UWM in Olsztyn.

Variable features

- properties that differentiate statistical units in a given population.
- example: UWM students - variable features: age, sex, type of secondary school completed, eye color, height.

Important:

- only variable features are subject to observation,
- a constant feature in one population may be a variable feature in another population.

Example: UWM students have an ID card issued by UWM. Students of all universities in Poland have ID cards issued by different institutions.

Division of variable features:

- measurable (quantitative) features - can be expressed as a number with a specific unit of measurement.
- non-measurable (qualitative) features - described verbally, representing certain categories.

Example: student population. Measurable features: age, weight, height, number of absences. Non-measurable features: sex, eye color, field of study.

For practical reasons, numerical codes are often assigned to non-measurable features. However, they should not be confused with measurable features. E.g. 1 - primary education, 2 - vocational education, etc.

Division of measurable features:

- continuous - can take any value within a defined interval, e.g. height, age, apartment area.
- discrete - can take specific (discrete) numerical values without intermediate values, e.g. number of people in a household, number of employees in a given company.

Discrete features usually have integer values, although this is not always required, e.g. the number of full-time positions in a company (including part-time positions).

2.4.7 Scales

Measurement scale

- a system that allows the results of statistical measurements to be organized in a certain way.
- division:
 - nominal scale,
 - ordinal scale,
 - interval scale,
 - ratio scale.

Nominal scale

- a scale in which a statistical unit is classified into a specific category.
- values in this scale have no ordering.
- example:

Religion	Code
Christianity	1
Islam	2
Buddhism	3

Ordinal scale

- values have a clearly defined order, but distances between them are not given,
- allows elements to be ranked.
- examples:

Education	Code
Primary	1
Secondary	2
Higher	3

Income	Code
Low	1
Medium	2
High	3

Interval scale

- feature values are expressed through specific numerical values,
- allows comparison of units (something is larger or smaller),
- it is not possible to examine ratios (determining how many times a given value is larger or smaller than another).
- example:

City	Temperature in $^{\circ}C$	Temperature in $^{\circ}F$
Warsaw	15	59
Olsztyn	10	50
Gdańsk	5	41
Szczecin	20	68

Ratio scale

- values are expressed through numerical values,
- it is possible to determine the smaller or larger relationship between values,
- it is possible to determine the ratio (quotient) between values,
- an absolute zero exists.
- example:

Product	Price in PLN
Bread	3
Butter	8
Pears	5

2.5 Types of statistical studies

- complete study - covers all units of the statistical population.
 - statistical census,
 - ongoing registration,

- statistical reporting.
- partial studies - only a part of the population is observed. Conducted when a complete study is impractical or impossible.
 - monographic method,
 - representative method.

2.6 Stages of a statistical study

- designing and organizing the study: establishing the purpose, subject, object, scope, source, and duration of the study;
- statistical observation;
- processing statistical material: quality control of statistical material, grouping of obtained data, presentation of data results;
- statistical analysis.

2.7 Analysis of existing data

Analysis of existing data - the process of processing data in order to obtain useful information and conclusions from them. Depending on the type of data and the problems posed, this may involve the use of statistical, exploratory, and other methods.

Using existing data is an example of non-reactive research - methods of studying social behavior that do not influence those behaviors. Such data include: documents, archives, reports, chronicles, censuses, parish records, diaries, memoirs, internet blogs, audio memoirs, oral history archives, and others. (Wikipedia)

Existing data can be classified according to (Makowska ed. 2013):

- Nature: Quantitative, Qualitative
- Form: Processed data, Raw data
- Method of creation: Primary, Secondary
- Dynamics: Continuous event registration, Registration at time intervals, One-time registration
- Level of objectivity: Objective, Subjective
- Source of origin: Public data, Private data

Data analysis is a process of inspecting, organizing, transforming, and modeling data in order to obtain useful information, draw conclusions, and support the decision-making process. Data analysis has many aspects and approaches, encompassing various techniques under different names, in different business, scientific, and social domains. A practical approach to defining

data is that data are numbers, characters, images, or other recording methods, in a form that can be evaluated to determine or make a decision about a specific action. Many people believe that data in itself has no meaning - only processed and interpreted data becomes information.

2.8 Data analysis process

Analysis refers to breaking down the entirety of information into its distinct components for individual examination. Data analysis is the process of obtaining raw data and transforming it into information useful for decision-making by users. Data is collected and analyzed to answer questions, test hypotheses, or disprove theories. There are several phases that can be distinguished in the data analysis process. The phases are iterative, as feedback from subsequent phases may cause additional work in earlier phases.

2.8.1 Defining requirements

Before proceeding with data analysis, the quality requirements for data must be precisely defined. The input data to be analyzed are determined based on the requirements of those directing the analysis or clients (who will use the final product of the analysis). The general type of entity from which data will be collected is referred to as the experimental unit (e.g. a person or a population of people). Data can be numerical or categorical (i.e. text labels). The requirements definition phase should answer 2 fundamental questions:

- what do we want to measure?
- how do we want to measure it?

2.8.2 Data collection

Data is collected from various sources. Requirements regarding the type and quality of data may be communicated by analysts to “data custodians,” such as information technology staff within an organization. Data may also be collected automatically from various types of sensors in the environment - such as traffic cameras, satellites, and devices recording images, sound, and physical parameters. Another method is obtaining data through interviews, gathering from online sources, or directly from documentation.

2.8.3 Data processing

Collected data must be processed or organized in a logical manner for analysis. For example, they may be placed in tables for further analysis - in a spreadsheet or other software. Data cleaning: After the processing and organizing phase, data may be incomplete, contain duplicates,

or contain errors. The need for data cleaning arises from problems related to data entry and storage. Data cleaning is the process of preventing and correcting detected errors. Common tasks include record matching, identifying inaccuracies, overall review of existing data quality, removing duplicates, and column segmentation. It is also extremely important to pay attention to data whose values are above or below previously established thresholds (extremes).

2.8.4 Proper data analysis

There are several methods that can be used for this purpose, for example data mining, business intelligence, data visualization, or exploratory research. The latter method is a way of analyzing datasets to determine their distinct characteristics. In this way, data can be used to test the original hypothesis. Descriptive statistics is another method for analyzing collected information. Data is examined to find its most important features. In descriptive statistics, analysts use several basic tools - the mean or average of a set of numbers can be used. This helps determine the overall trend, although it does not provide great accuracy when assessing the overall picture of collected data. In this phase, modeling and the creation of mathematical formulas also take place - they are applied to identify relationships between variables, such as correlation or causation.

2.8.5 Reporting and distribution of results

This phase involves determining in what form to communicate results. The analyst may consider various data visualization techniques to clearly and effectively convey the findings of the analysis to the audience. Data visualization uses graphical forms such as charts and tables. Tables are useful for users who can search for specific records, while charts (e.g. bar charts or line charts) provide a quantitative view of the analyzed dataset.

2.9 Where to get data?

Free data repositories:

- Local Data Bank of the Central Statistical Office (GUS) - [link](#)
- Open Data - [link](#)
- World Bank - [link](#)

2.10 “Tidy data” concept

The tidy data concept:

- WICKHAM, Hadley . Tidy Data. Journal of Statistical Software, [S.l.], v. 59, Issue 10, p. 1 - 23, sep. 2014. ISSN 1548-7660. Date accessed: 25 oct. 2018. doi:<http://dx.doi.org/10.18637/jss.v059.i10>.

2.10.1 “Tidy data” principles

Ideal data is presented in a table:

Name	Age	Height	Eye color
Adam	26	167	Brown
Sylwia	34	164	Hazel
Tomasz	42	183	Blue

What should we pay attention to?

- one observation (statistical unit) = one row in the table/matrix/data frame
- values of a given feature are found in columns
- one type/kind of observation in one table/matrix/data frame

2.10.2 Examples of untidy data

Name	Age	Height	Brown	Blue	Hazel
Adam	26	167	1	0	0
Sylwia	34	164	0	0	1
Tomasz	42	183	0	1	0

Column headers must correspond to features, not to variable values.

2.11 A few tips for good presentations

Edward Tufte, professor at Yale

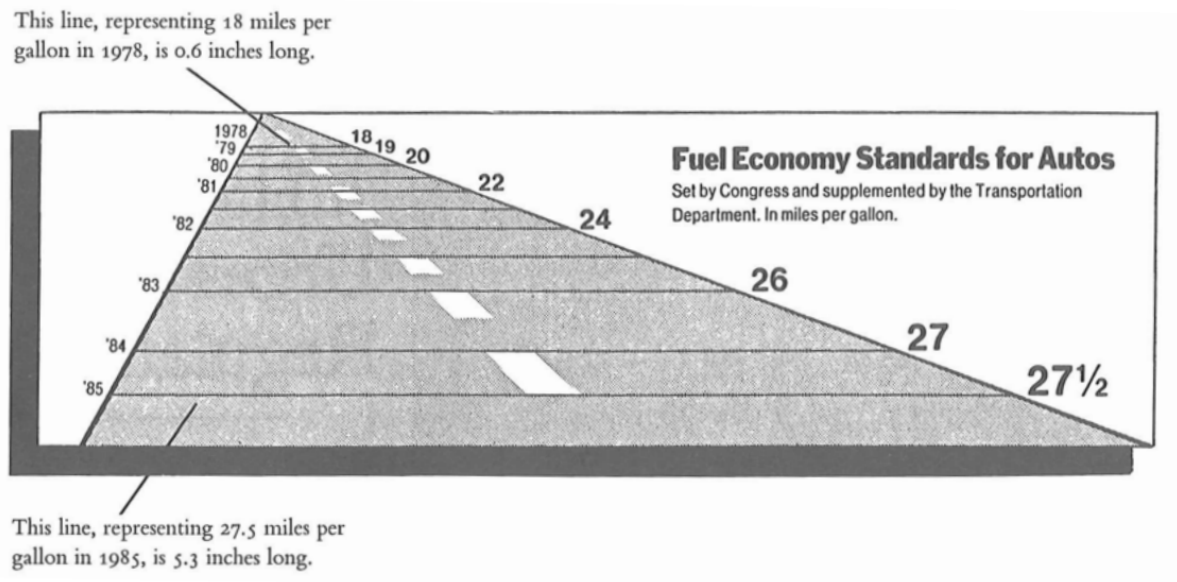
1. Present data richly.
2. Do not hide data, show the truth.
3. Do not use junk charts.
4. Show the variability of data, do not design it.

5. A chart should have the lowest possible lie factor.
6. PowerPoint is evil!

2.11.1 Lie factor

- the ratio of the effect visible on the chart to the effect indicated by the data on which the chart was drawn.

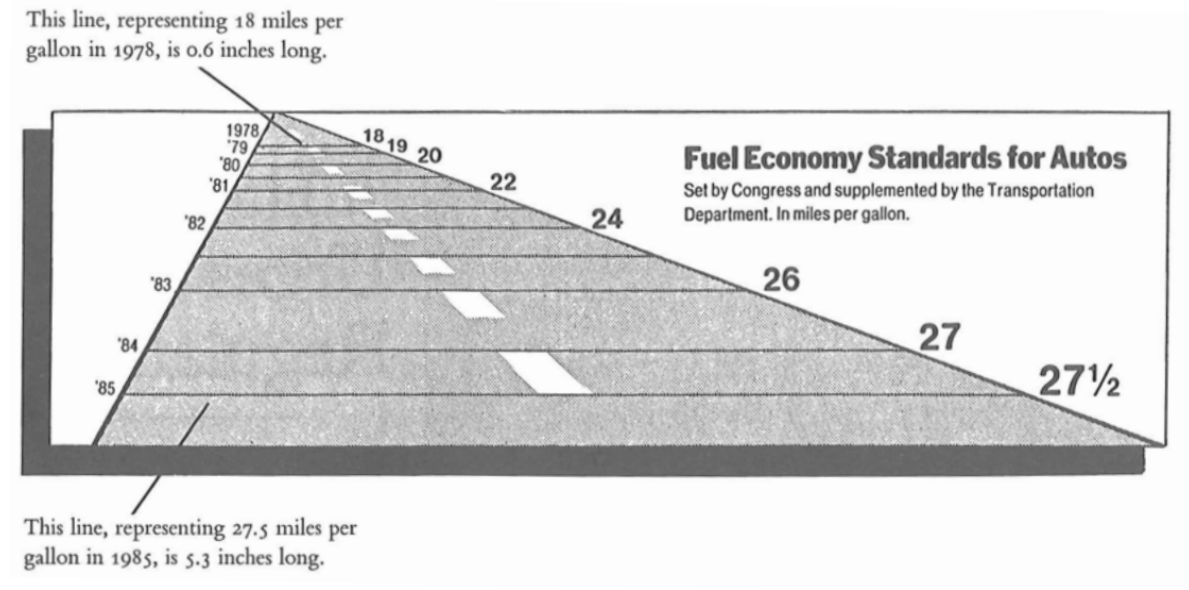
2.11.2 Lie factor



[Tufte, 1991] Edward Tufte, The Visual Display of Quantitative Information, Second Edition, Graphics Press, USA, 1991, p. 57 – 69.

$$\text{LieFactor} = \frac{\text{size of the effect visible on the chart}}{\text{size of the effect resulting from the data}}$$

$$\text{effect size} = \frac{|\text{second value} - \text{first value}|}{\text{first value}}$$



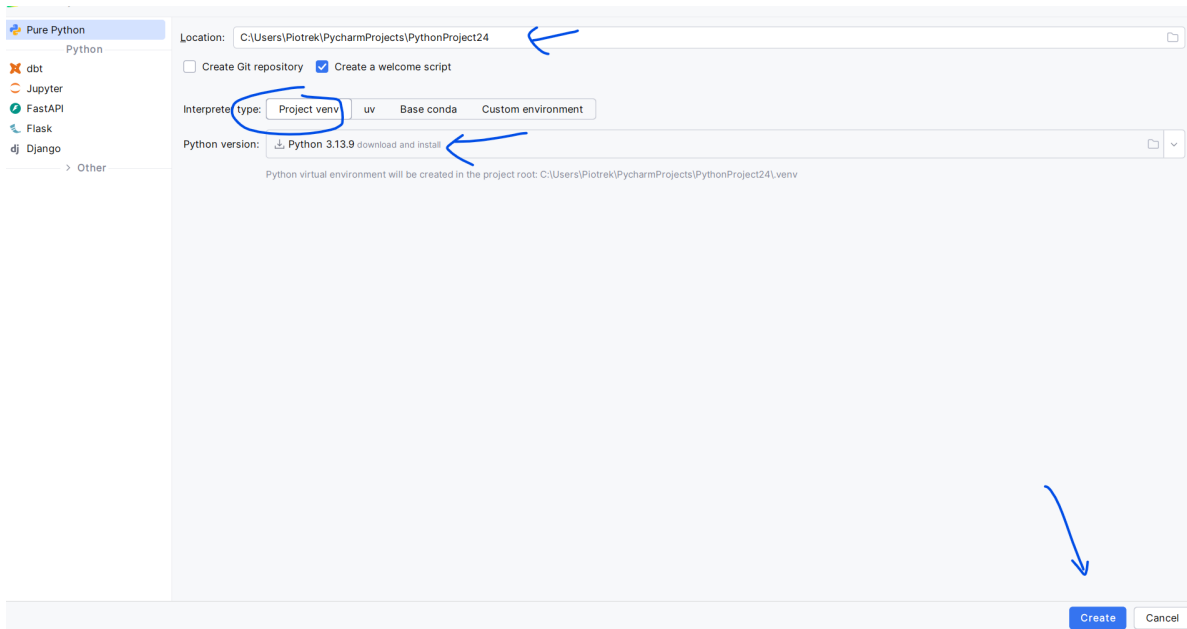
$$\text{LieFactor} = \frac{\frac{5.3-0.6}{0.6}}{\frac{27.5-18}{18}} \approx 14.8$$

2.12 References

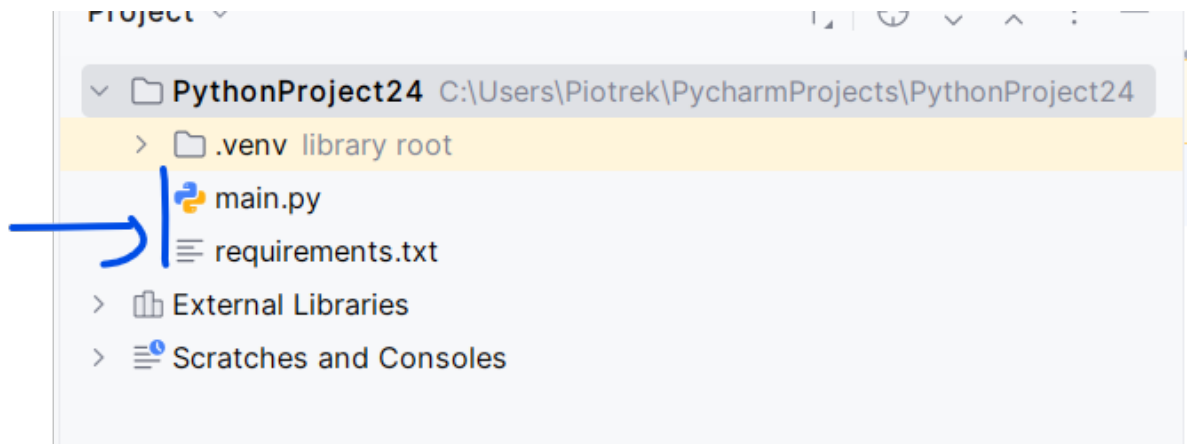
- <https://pl.wikipedia.org/wiki/Wizualizacja>
- https://mfiles.pl/pl/index.php/Analiza_danych, accessed online 1.04.2019.
- Walesiak M., Gatnar E., Statystyczna analiza danych z wykorzystaniem programu R, PWN, Warszawa, 2009.
- Wasilewska E., Statystyka opisowa od podstaw, Podręcznik z zadaniami, Wydawnictwo SGGW, Warszawa, 2009.
- https://en.wikipedia.org/wiki/Cognitive_reflection_test, accessed online 20.03.2023.
- <https://qlikblog.pl/edward-tufte-dobre-praktyki-prezentacji-danych/>, accessed online 20.03.2023.

3 Configuration

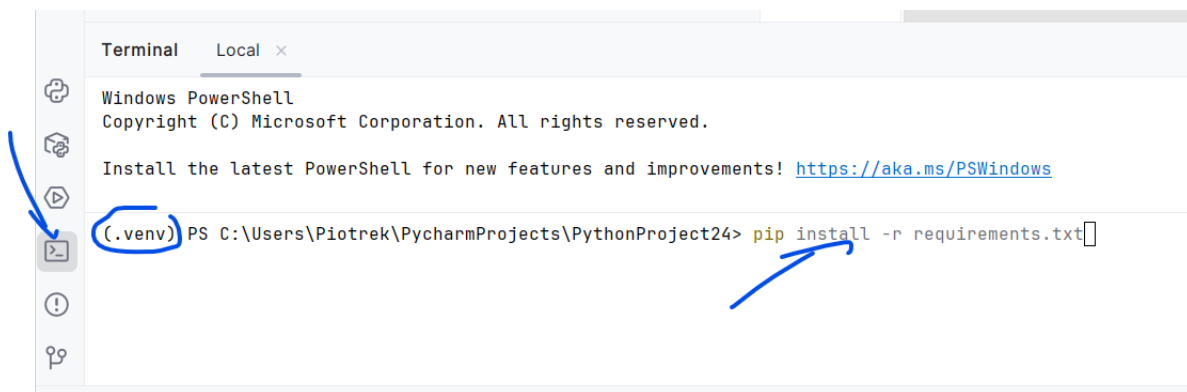
1. In the first step, we create a new project in the chosen location. We select venv and one of the Python versions, e.g. 3.13. Finally, click the Create button.



2. We add the requirements file <https://piojas.pl/wp-content/uploads/2026/02/requirements.txt> to the project's root directory. The file name is important.



3. We open the terminal in PyCharm and type the command: `pip install -r requirements.txt`



Part II

NumPy

4 NumPy - Getting Started

NumPy is a Python library for scientific computing.

Applications:

- linear algebra
- advanced mathematical (numerical) computations
- numerical integration
- solving equations

4.1 Importing the NumPy library

```
import numpy as np
```

The fundamental object in the NumPy library is an N-dimensional array called `ndarray`. Each element of the array is treated as a `dtype` type.

```
np.array(object, dtype=None, *, copy=True, order='K', subok=False, ndmin=0, like=None)
```

- `object` - an object (or sequence) to be converted into an array
- `dtype` - type
- `copy` - whether objects should be copied, default is `True`
- `order` - memory layout: C (rows), F (columns), A, K
- `subok` - handled by subclasses (if `True`), default is `False`
- `ndmin` - minimum number of dimensions of the array
- `like` - creation based on a reference array

```
import numpy as np
```

```
a = np.array([1, 2, 3])
```

①

```
print("a:", a)
```

```
print("type of a:", type(a))
```

②

```
b = np.array([1, 2, 3.0])
```

③

```

print("b:", b)
c = np.array([[1, 2], [3, 4]])
print("c:", c)
d = np.array([1, 2, 3], ndmin=2)
print("d:", d)
e = np.array([1, 2, 3], dtype=complex)
print("e:", e)
f = np.array(np.asmatrix('1 2; 3 4'))
print("f:", f)
g = np.array(np.asmatrix('1 2; 3 4'), subok=True)
print("g:", g)
print(type(g))

```

- ① Creating an array with default settings.
- ② Checking the type.
- ③ One of the elements is of a different type. Therefore, upcasting to the “largest” type occurs.
- ④ Here we get a 2x2 array.
- ⑤ In this line we get an array with dimensions 1x3 (shape (1, 3)).
- ⑥ Forcing the **complex** type — a type with a wider range.
- ⑦ Using the matrix subtype.
- ⑧ Preserving the matrix type.

```

a: [1 2 3]
type of a: <class 'numpy.ndarray'>
b: [1. 2. 3.]
c: [[1 2]
     [3 4]]
d: [[1 2 3]]
e: [1.+0.j 2.+0.j 3.+0.j]
f: [[1 2]
     [3 4]]
g: [[1 2]
     [3 4]]
<class 'numpy.matrix'>

```

5 List vs Array

```
import numpy as np
import time

start_time = time.time()
my_arr = np.arange(1000000)
my_list = list(range(1000000))
start_time = time.time()
my_arr2 = my_arr * 2
print("--- %s seconds ---" % (time.time() - start_time))
start_time = time.time()
my_list2 = [x * 2 for x in my_list]
print("--- %s seconds ---" % (time.time() - start_time))
```

```
--- 0.0020012855529785156 seconds ---
--- 0.036900997161865234 seconds ---
```


6 Array Attributes

Attribute	Description
shape	tuple with information about the number of elements for each dimension
size	total number of elements in the array
ndim	number of dimensions of the array
nbytes	number of bytes the array occupies in memory
dtype	data type stored in the array (e.g. <code>int64</code> , <code>float64</code>)

```
import numpy as np

tab1 = np.array([2, -3, 4, -8, 1])
print("type:", type(tab1))
print("shape:", tab1.shape)
print("size:", tab1.size)
print("ndim:", tab1.ndim)
print("nbytes:", tab1.nbytes)
print("dtype:", tab1.dtype)
```

```
type: <class 'numpy.ndarray'>
shape: (5,)
size: 5
ndim: 1
nbytes: 40
dtype: int64
```

```
import numpy as np

tab2 = np.array([[2, -3], [4, -8]])
print("type:", type(tab2))
print("shape:", tab2.shape)
print("size:", tab2.size)
```

```
print("ndim:", tab2.ndim)
print("nbytes:", tab2.nbytes)
print("dtype:", tab2.dtype)
```

```
type: <class 'numpy.ndarray'>
shape: (2, 2)
size: 4
ndim: 2
nbytes: 32
dtype: int64
```

NumPy does not support ragged arrays. The following code will produce an error:

```
import numpy as np

tab3 = np.array([[2, -3], [4, -8, 5], [3]])
```

7 Data Types

Integer types	int,int8,int16,int32,int64
Unsigned integer types	uint,uint8,uint16,uint32,uint64
Boolean type	bool
Floating-point types	float, float16, float32, float64, float128
Complex floating-point types	complex, complex64, complex128, complex256
String	str

```
import numpy as np

tab = np.array([[2, -3], [4, -8]])
print(tab)
tab2 = np.array([[2, -3], [4, -8]], dtype=int)
print(tab2)
tab3 = np.array([[2, -3], [4, -8]], dtype=float)
print(tab3)
tab4 = np.array([[2, -3], [4, -8]], dtype=complex)
print(tab4)
```

```
[[ 2 -3]
 [ 4 -8]]
[[ 2 -3]
 [ 4 -8]]
[[ 2. -3.]
 [ 4. -8.]]
[[ 2.+0.j -3.+0.j]
 [ 4.+0.j -8.+0.j]]
```

8 Creating Arrays

`np.array` - arguments are cast to an array (anything iterable) - it's worth checking the size/shape

```
import numpy as np

tab = np.array([2, -3, 4])
print(tab)
print("size:", tab.size)
tab2 = np.array((4, -3, 3, 2))
print(tab2)
print("size:", tab2.size)
tab3 = np.array({3, 3, 2, 5, 2})
print(tab3)
print("size:", tab3.size)
tab4 = np.array({"pl": 344, "en": 22})
print(tab4)
print("size:", tab4.size)
```

```
[ 2 -3  4]
size: 3
[ 4 -3  3  2]
size: 4
{2, 3, 5}
size: 1
{'pl': 344, 'en': 22}
size: 1
```

`np.zeros` - creates an array filled with zeros

```
import numpy as np

tab = np.zeros(4)
print(tab)
print(tab.shape)
```

```

tab2 = np.zeros([2, 3])
print(tab2)
print(tab2.shape)
tab3 = np.zeros([2, 3, 4])
print(tab3)
print(tab3.shape)

```

```

[0. 0. 0. 0.]
(4,)
[[0. 0. 0.]
 [0. 0. 0.]]
(2, 3)
[[[0. 0. 0. 0.]
   [0. 0. 0. 0.]
   [0. 0. 0. 0.]]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
(2, 3, 4)

```

`np.ones` - creates an array filled with ones (this is not the equivalent of an identity matrix!)

```

import numpy as np

tab = np.ones(4)
print(tab)
print(tab.shape)
tab2 = np.ones([2, 3])
print(tab2)
print(tab2.shape)
tab3 = np.ones([2, 3, 4])
print(tab3)
print(tab3.shape)

```

```

[1. 1. 1. 1.]
(4,)
[[1. 1. 1.]
 [1. 1. 1.]]
(2, 3)
[[[1. 1. 1. 1.]
   [1. 1. 1. 1.]
   [1. 1. 1. 1.]]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]]

```

```

[1.  1.  1.  1.]
[1.  1.  1.  1.]]

[[1.  1.  1.  1.]
 [1.  1.  1.  1.]
 [1.  1.  1.  1.]]]
(2, 3, 4)

```

`np.diag` - creates an array corresponding to a diagonal matrix

```

import numpy as np

print("tab0")
tab0 = np.diag([3, 4, 5])
print(tab0)
print("tab1")
tab1 = np.array([[2, 3, 4], [3, -4, 5], [3, 4, -5]])
print(tab1)
tab2 = np.diag(tab1)
print("tab2")
print(tab2)
tab3 = np.diag(tab1, k=1)
print("tab3")
print(tab3)
print("tab4")
tab4 = np.diag(tab1, k=-2)
print(tab4)
print("tab5")
tab5 = np.diag(np.diag(tab1))
print(tab5)

```

```

tab0
[[3 0 0]
 [0 4 0]
 [0 0 5]]
tab1
[[ 2  3  4]
 [ 3 -4  5]
 [ 3  4 -5]]
tab2
[ 2 -4 -5]
tab3

```

```
[3 5]
tab4
[3]
tab5
[[ 2  0  0]
 [ 0 -4  0]
 [ 0  0 -5]]
```

`np.arange` - array filled with evenly spaced values

Syntax: `numpy.arange([start, ]stop, [step, ]dtype=None)`

Works similarly to the `range` function, but allows fractional numbers.

```
import numpy as np

a = np.arange(3)
print(a)
b = np.arange(3.0)
print(b)
c = np.arange(3, 7)
print(c)
d = np.arange(3, 11, 2)
print(d)
e = np.arange(0, 1, 0.1)
print(e)
f = np.arange(3, 11, 2, dtype=float)
print(f)
g = np.arange(3, 10, 2)
print(g)
```

```
[0 1 2]
[0. 1. 2.]
[3 4 5 6]
[3 5 7 9]
[0.  0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9]
[3. 5. 7. 9.]
[3 5 7 9]
```

`np.linspace` - array filled with evenly spaced values on a linear scale

`np.linspace(start, stop, num)` generates an array of evenly spaced values. For example, `np.linspace(2.0, 3.0, num=5)` produces 2, 2.25, 2.5, 2.75, 3 — the interval is divided

into `num - 1` equal parts. Setting `endpoint=False` removes the last point (from the right side), and the interval is then divided into `num` equal parts instead. The `dtype` parameter forces the output type — typically set to `int`, in which case the results may have the fractional part truncated.

```
import numpy as np

a = np.linspace(2.0, 3.0, num=5)
print(a)
b = np.linspace(2.0, 3.0, num=5, endpoint=False)
print(b)
c = np.linspace(10, 20, num=4)
print(c)
d = np.linspace(10, 20, num=4, dtype=int)
print(d)
```

```
[2.  2.25 2.5  2.75 3.  ]
[2.  2.2 2.4 2.6 2.8]
[10.          13.33333333 16.66666667 20.          ]
[10 13 16 20]
```

`np.logspace` - array filled with values on a logarithmic scale

Syntax: `numpy.logspace(start, stop, num=50, endpoint=True, base=10.0, dtype=None, axis=0)`

The function `np.logspace(2.0, 3.0, num=4)` generates 4 numbers spaced evenly on a logarithmic scale between 10^2 and 10^3 . The default base is 10. First, the exponents are computed as equally spaced points in the interval $[2.0, 3.0]$, which gives approximately 2, 2.33, 2.66, and 3.0. Then each exponent is used as the power of 10, producing the output values $10^2 = 100$, $10^{2.33} \approx 215.44$, $10^{2.66} \approx 464.16$, and $10^3 = 1000$. An important consequence is that while the exponents are uniformly distributed, the resulting values are not — the distances between consecutive output numbers grow larger as the values increase. This is the key distinction between `np.logspace` and `np.linspace`: the former creates equal spacing in log-scale (i.e. between the exponents), not in the actual output values.

```
import numpy as np

a = np.logspace(2.0, 3.0, num=4)
print(a)
b = np.logspace(2.0, 3.0, num=4, endpoint=False)
print(b)
```



```
c = np.logspace(2.0, 3.0, num=4, base=2.0)
print(c)
```

```
[ 100.          215.443469   464.15888336 1000.          ]
[100.          177.827941   316.22776602 562.34132519]
[4.           5.0396842   6.34960421 8.           ]
```

`np.empty` - empty (uninitialized) array - specific values are not “guaranteed”

```
import numpy as np

a = np.empty(3)
print(a)
b = np.empty(3, dtype=int)
print(b)
```

```
[0.  1.  2.]
[          0 4607182418800017408 4611686018427387904]
```

`np.identity` - array resembling an identity matrix

`np.eye` - array with ones on the diagonal (remaining values are zeros)

```
import numpy as np

print("a")
a = np.identity(4)
print(a)
print("b")
b = np.eye(4, k=1)
print(b)
print("c")
c = np.eye(4, k=2)
print(c)
print("d")
d = np.eye(4, k=-1)
print(d)
```

```
a
[[1.  0.  0.  0.]
```

```
[0. 1. 0. 0.]  
[0. 0. 1. 0.]  
[0. 0. 0. 1.]]
```

b

```
[[0. 1. 0. 0.]  
 [0. 0. 1. 0.]  
 [0. 0. 0. 1.]  
 [0. 0. 0. 0.]]
```

c

```
[[0. 0. 1. 0.]  
 [0. 0. 0. 1.]  
 [0. 0. 0. 0.]  
 [0. 0. 0. 0.]]
```

d

```
[[0. 0. 0. 0.]  
 [1. 0. 0. 0.]  
 [0. 1. 0. 0.]  
 [0. 0. 1. 0.]]
```

9 Indexing, “slicing”

```
import numpy as np

a = np.array([2, 5, -2, 4, -7, 8, 9, 11, -23, -4, -7, 16, 1])
print("1:", a[5])
print("2:", a[-2])
print("3:", a[3:6])
print("4:", a[:])
print("5:", a[0:-1])
print("6:", a[:5])
```

①
②
③
④
⑤
⑥

- ① Access the element at index 5.
- ② Access the second element from the end.
- ③ Access elements with indices from 3 to 5 (inclusive) — the principle of half-open intervals (closed on the left, open on the right).
- ④ Access all elements.
- ⑤ Access all elements except the last one.
- ⑥ Access from the beginning up to the element at index 4.

```
1: 8
2: 16
3: [ 4 -7  8]
4: [  2  5 -2  4 -7  8  9 11 -23 -4 -7 16  1]
5: [  2  5 -2  4 -7  8  9 11 -23 -4 -7 16]
6: [ 2  5 -2  4 -7]
```

```
import numpy as np

print("1:", a[4:])
print("2:", a[4:-1])
print("3:", a[4:10:2])
print("4:", a[::-1])
print("5:", a[:2])
print("6:", a[::-2])
```

①
②
③
④
⑤
⑥

- ① Access elements from index 4 to the end.
- ② Access elements from index 4 to the end, excluding the last one.
- ③ Access elements whose indices form an arithmetic sequence from 4 to 10 (including 4, but excluding 10) with a step of 2.
- ④ Access elements in reverse order (from end to beginning).
- ⑤ Access elements at even indices from the beginning.
- ⑥ Access every second element, counting from the end (reversed order).

```
1: [ -7   8   9  11 -23  -4  -7  16   1]
2: [ -7   8   9  11 -23  -4  -7  16]
3: [ -7   9 -23]
4: [  1  16  -7  -4 -23  11   9   8  -7   4  -2   5   2]
5: [  2  -2  -7   9 -23  -7   1]
6: [  1  -7 -23   9  -7  -2   2]
```

```
import numpy as np

a = np.array([[3, 4, 5], [-3, 4, 8], [3, 2, 9]])
b = a[:, 1:]
print(b)
print(np.shape(b))
c = a[1]
print(c)
print(np.shape(c))
d = a[1, :]
print(d)
print(np.shape(d))
```

```
[[4 5]
 [4 8]]
(2, 2)
[-3  4  8]
(3,)
[-3  4  8]
(3,)
```

```
import numpy as np

a = np.array([[3, 4, 5], [-3, 4, 8], [3, 2, 9]])
e = a[1:2, :]
print(e)
```

```

print(np.shape(e))
f = a[:, :2]
print(f)
print(np.shape(f))
g = a[1, :2]
print(g)
print(np.shape(g))
h = a[1:2, :2]
print(h)
print(np.shape(h))

```

```

[[-3  4  8]]
(1, 3)
[[ 3  4]
 [-3  4]
 [ 3  2]]
(3, 2)
[-3  4]
(2,)
[[-3  4]]
(1, 2)

```

Note — “slicing” creates a so-called **view**, not a copy. Changes to the view affect the original array and vice versa.

```

import numpy as np

a = np.array([[3, 4, 5], [-3, 4, 8], [3, 2, 9]])
b = a[1:2, 1:]
print(b)
a[1][1] = 9
print(a)
print(b)
b[0][0] = -11
print(a)
print(b)

```

```

[[4 8]]
[[ 3  4  5]
 [-3  9  8]
 [ 3  2  9]]

```

```
[[9 8]]
[[ 3  4  5]
 [-3 -11  8]
 [ 3  2  9]]
[[-11  8]]
```

Solution — use the `.copy()` method:

```
import numpy as np

a = np.array([[3, 4, 5], [-3, 4, 8], [3, 2, 9]])
b = a[1:2, 1:].copy()
print(b)
a[1][1] = 9
print(a)
print(b)
b[0][0] = -11
print(a)
print(b)
```

```
[[4 8]]
[[ 3  4  5]
 [-3  9  8]
 [ 3  2  9]]
[[4 8]]
[[ 3  4  5]
 [-3  9  8]
 [ 3  2  9]]
[[-11  8]]
```

Fancy indexing (indexing with an array of indices)

```
import numpy as np

a = np.array([2, 5, -2, 4, -7, 8, 9, 11, -23, -4, -7, 8, 1])
b = a[np.array([1, 3, 7])]
print(b)
c = a[[1, 3, 7]]
print(c)
```

```
[ 5  4 11]
[ 5  4 11]
```

Boolean indexing (boolean masks)

```
import numpy as np

a = np.array([2, 5, -2, 4, -7, 8, 9, 11, -23, -4, -7, 8, 1])
b = a > 0
print(b)
c = a[a > 0]
print(c)
d = a[(a > 5) & (a%2 !=0)] # the & operator stands for AND
print(d)
e = a[(a > 5) | (a%2 !=0)] # the | operator stands for OR
print(e)
f = a[(a > 5) ^ (a%2 !=0)] # the ^ operator stands for XOR
print(f)
g = a[~(a > 0)]
print(g)
```

```
[ True  True False  True False  True  True  True False False False  True
  True]
[ 2  5  4  8  9 11  8  1]
[ 9 11]
[ 5 -7  8  9 11 -23 -7  8  1]
[ 5 -7  8 -23 -7  8  1]
[-2 -7 -23 -4 -7]
```

Note — fancy and boolean indexing return a copy, not a view (unlike “slicing”).

```
import numpy as np

a = np.array([2, 5, -2, 4, -7, 8, 9, 11, -23, -4, -7, 8, 1])
b = a[a > 0]
print(b)
b[0] = -5
print(a)
print(b)
a[1] = 20
print(a)
print(b)
```

```
[ 2  5  4  8  9 11  8  1]
```

```

[ 2  5 -2  4 -7  8  9 11 -23 -4 -7  8  1]
[-5 5  4  8  9 11  8  1]
[ 2 20 -2  4 -7  8  9 11 -23 -4 -7  8  1]
[-5 5  4  8  9 11  8  1]

```


10 Modifying shape and size

```
import numpy as np

print("a")
a = np.array([[3, 4, 5], [-3, 4, 8], [3, 2, 9]])
print(a)
print("b")
b = np.reshape(a, (1, 9))
print(b)
print("c")
c = a.reshape(9)
print(c)
```

```
a
[[ 3  4  5]
 [-3  4  8]
 [ 3  2  9]]
b
[[ 3  4  5 -3  4  8  3  2  9]]
c
[ 3  4  5 -3  4  8  3  2  9]
```

```
import numpy as np

print("a")
a = np.array([[3, 4, 5], [-3, 4, 8], [3, 2, 9]])
print(a)
print("d")
d = a.flatten()    # always returns a copy
print(d)
print("e")
e = a.ravel()      # returns a view (shared memory with original)
print(e)
print("f")
```

```
f = np.ravel(a)
print(f)
```

```
a
[[ 3  4  5]
 [-3  4  8]
 [ 3  2  9]]
d
[ 3  4  5 -3  4  8  3  2  9]
e
[ 3  4  5 -3  4  8  3  2  9]
f
[ 3  4  5 -3  4  8  3  2  9]
```

```
import numpy as np

print("g")
g = np.array([[1, 3, 4]])
print(g)
print("h")
h = np.squeeze(g)
print(h)
print("i")
a = np.array([[3, 4, 5], [-3, 4, 8], [3, 2, 9]])
print(a)
i = a.T
print(i)
print("j")
j = np.transpose(a)
print(j)
```

```
g
[[1 3 4]]
h
[1 3 4]
i
[[ 3  4  5]
 [-3  4  8]
 [ 3  2  9]]
[[ 3 -3  3]
 [ 4  4  2]]
```

```

[ 5  8  9]]
j
[[ 3 -3  3]
 [ 4  4  2]
 [ 5  8  9]]

```

```

import numpy as np

print("n")
n = np.array([3, -4, 5, -2])
print(n)
print("k")
k = np.hstack((n, n, n))
print(k)
print("l")
l = np.vstack((n, n, n))
print(l)
print("m")
m = np.dstack((n, n, n))
print(m)

```

```

n
[ 3 -4  5 -2]
k
[ 3 -4  5 -2  3 -4  5 -2  3 -4  5 -2]
l
[[ 3 -4  5 -2]
 [ 3 -4  5 -2]
 [ 3 -4  5 -2]]
m
[[[ 3  3  3]
  [-4 -4 -4]
  [ 5  5  5]
  [-2 -2 -2]]]

```

```

import numpy as np

a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6]])
print("r1")
r1 = np.concatenate((a, b))      # axis=0 is the default
print(r1)

```

```

print("r2")
r2 = np.concatenate((a, b), axis=0) # same as above (explicit)
print(r2)
print("r3")
r3 = np.concatenate((a, b.T), axis=1)
print(r3)
print("r4")
r4 = np.concatenate((a, b), axis=None)
print(r4)

```

```

r1
[[1 2]
 [3 4]
 [5 6]]
r2
[[1 2]
 [3 4]
 [5 6]]
r3
[[1 2 5]
 [3 4 6]]
r4
[1 2 3 4 5 6]

```

```

import numpy as np

a = np.array([[1, 2], [3, 4]])
print("r1")
r1 = np.resize(a, (2, 3))
print(r1)
print("r2")
r2 = np.resize(a, (1, 4))
print(r2)
print("r3")
r3 = np.resize(a, (2, 4))
print(r3)

```

```

r1
[[1 2 3]
 [4 1 2]]
r2

```

```
[[1 2 3 4]]
r3
[[1 2 3 4]
 [1 2 3 4]]
```

```
import numpy as np

a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6]])
print("r1")
r1 = np.append(a, b)
print(r1)
print("r2")
r2 = np.append(a, b, axis=0)
print(r2)
```

```
r1
[1 2 3 4 5 6]
r2
[[1 2]
 [3 4]
 [5 6]]
```

```
import numpy as np

a = np.array([[1, 2], [3, 7]])
print("r1")
r1 = np.insert(a, 1, 4)
print(r1)
print("r2")
r2 = np.insert(a, 2, 4)
print(r2)
print("r3")
r3 = np.insert(a, 1, 4, axis=0)
print(r3)
print("r4")
r4 = np.insert(a, 1, 4, axis=1)
print(r4)
```

```
r1
[1 4 2 3 7]
```

```
r2
[1 2 4 3 7]
r3
[[1 2]
 [4 4]
 [3 7]]
r4
[[1 4 2]
 [3 4 7]]
```

```
import numpy as np

a = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
print("r1")
r1 = np.delete(a, 1, axis=1)
print(r1)
print("r2")
r2 = np.delete(a, 2, axis=0)
print(r2)
```

```
r1
[[ 1  3  4]
 [ 5  7  8]
 [ 9 11 12]]
r2
[[1 2 3 4]
 [5 6 7 8]]
```

11 Broadcasting

The variants discussed below are illustrative examples.

Variant 1 — scalar-array — performing an operation on every element of the array

When a scalar is combined with an array, NumPy broadcasts the scalar to match the array's shape, then performs the operation element-wise. In the code above, array **a** has shape 3×2 and contains $\begin{bmatrix} 1 & 2 \\ 5 & 6 \\ 9 & 10 \end{bmatrix}$. The expression **a** + 4 conceptually expands the scalar 4 into a 3×2 array filled with 4s, so the result **b** is $\begin{bmatrix} 5 & 6 \\ 9 & 10 \\ 13 & 14 \end{bmatrix}$. The same principle applies to **2** ** **a**, where the scalar 2 is broadcast across every element of **a**, producing $\begin{bmatrix} 2 & 4 \\ 32 & 64 \\ 512 & 1024 \end{bmatrix}$ — each element is computed as 2 raised to the corresponding value in **a**.

```
import numpy as np

a = np.array([[1, 2], [5, 6], [9, 10]])
b = a + 4
print(b)
c = 2 ** a
print(c)
```

```
[[ 5  6]
 [ 9 10]
 [13 14]]
[[ 2  4]
 [32 64]
 [512 1024]]
```

Variant 2 — two arrays — “when one of the arrays can be expanded” (both dimensions are equal or one of them is equal to 1)

When two arrays of different but compatible shapes are combined, NumPy stretches the smaller one to match the larger. Array **a** has shape 2×2 and **b** is a 1D array of shape (2,), which NumPy treats as a single row. In **a** + **b**, the row $[9, 2]$ is broadcast across both rows of **a**, giving $\begin{bmatrix} 10 & 4 \\ 14 & 8 \end{bmatrix}$. Similarly, **a** / **b** divides each row of **a** element-wise by $[9, 2]$, producing $\begin{bmatrix} 1/9 & 1 \\ 5/9 & 3 \end{bmatrix}$. Array **c** has shape 2×1 , meaning it acts as a column. In **a** + **c**, the value 4 is added across the entire first row and -2 across the second, yielding $\begin{bmatrix} 5 & 6 \\ 3 & 4 \end{bmatrix}$. Likewise, **c** /

`a` divides the column values by each element in the corresponding row, so the first row becomes $[4/1, 4/2]$ and the second $[-2/5, -2/6]$. The general rule is that dimensions are compatible when they are equal or one of them is 1 — the dimension of size 1 is then stretched to match the other.

```
import numpy as np

a = np.array([[1, 2], [5, 6]])
b = np.array([9, 2])
r1 = a + b
print(r1)
r2 = a / b
print(r2)
c = np.array([[4], [-2]])
r3 = a + c
print(r3)
r4 = c / a
print(r4)
```

```
[[10  4]
 [14  8]]
[[0.11111111  1.          ]
 [0.55555556  3.          ]]
[[5 6]
 [3 4]]
[[ 4.          2.          ]
 [-0.4         -0.33333333]]
```

Variant 3 — “column” and “row”

Array `a` is created as a row $[[5, 2, -3]]$ and then transposed with `.T`, turning it into a column of shape 3×1 . Array `b` is a 1D array of shape $(5,)$, treated by NumPy as a single row. When these two are combined, broadcasting stretches the column across 5 columns and the row across 3 rows, producing a 3×5 result in every case. In `a + b`, each value from the column $[5, 2, -3]$ is added to every element of the row $[3, -2, 1, 2, 4]$, giving $[[8, 3, 6, 7, 9], [5, 0, 3, 4, 6], [0, -5, -2, -1, 1]]$. The expression `b + a` yields the same result, since addition is commutative. For `a * b`, each column value is multiplied by every row value, producing $[[15, -10, 5, 10, 20], [6, -4, 2, 4, 8], [-9, 6, -3, -6, -12]]$. This column-times-row pattern is essentially an outer operation — every pair of elements from the two arrays is combined, and the result shape is determined by the lengths of the column and the row.


```
import numpy as np

a = np.array([[5, 2, -3]]).T
b = np.array([3, -2, 1, 2, 4])
print(a+b)
print(b+a)
print(a*b)
```

```
[[ 8  3  6  7  9]
 [ 5  0  3  4  6]
 [ 0 -5 -2 -1  1]]
[[ 8  3  6  7  9]
 [ 5  0  3  4  6]
 [ 0 -5 -2 -1  1]]
[[ 15 -10  5  10  20]
 [  6  -4  2   4   8]
 [ -9   6 -3  -6 -12]]
```

12 Universal functions (ufunc) and other mathematical functions

Universal functions (known as *ufuncs*) are among the most important tools in NumPy. They are functions that operate element-by-element on arrays, often implemented in C, which ensures high computational performance. Thanks to *ufuncs*, you can perform arithmetic, trigonometric, statistical, and logical operations on entire arrays in a simple and readable way, without the need to write loops in Python.

12.1 Basic arithmetic operations

NumPy automatically translates mathematical operators into the corresponding *ufuncs*. For example:

- `+` corresponds to `np.add`
- `-` corresponds to `np.subtract`
- `*` corresponds to `np.multiply`
- `/` corresponds to `np.divide`
- `**` corresponds to `np.power`

Example:

```
import numpy as np

A = np.array([1, 2, 3, 4])
B = np.array([10, 20, 30, 40])

# Element-by-element operations
sum_tab = np.add(A, B)          # same as A + B
diff_tab = np.subtract(B, A)    # same as B - A
mul_tab = np.multiply(A, 2)     # same as A * 2
pow_tab = np.power(A, 3)        # same as A ** 3

print("Sum:", sum_tab)
print("Difference:", diff_tab)
```

```
print("Multiplication by 2:", mul_tab)
print("Exponentiation:", pow_tab)
```

```
Sum: [11 22 33 44]
Difference: [ 9 18 27 36]
Multiplication by 2: [2 4 6 8]
Exponentiation: [ 1  8 27 64]
```

12.2 Trigonometric and related functions

NumPy offers a rich set of trigonometric functions:

- `np.sin`, `np.cos`, `np.tan` — basic functions,
- `np.arcsin`, `np.arccos`, `np.arctan` — inverse trigonometric functions,
- `np.sinh`, `np.cosh`, `np.tanh` — hyperbolic functions.

Example:

```
import numpy as np

x = np.linspace(0, np.pi, 5) # array [0, /4, /2, 3/4, ]
sin_values = np.sin(x)
cos_values = np.cos(x)

print("sin(x) values:", sin_values)
print("cos(x) values:", cos_values)
```

```
sin(x) values: [0.00000000e+00  7.07106781e-01  1.00000000e+00  7.07106781e-01
 1.22464680e-16]
cos(x) values: [ 1.00000000e+00  7.07106781e-01  6.12323400e-17 -7.07106781e-
01
-1.00000000e+00]
```

12.3 Exponential and logarithmic functions

- `np.exp` — exponential function,
- `np.log` — natural logarithm,
- `np.log10` — base-10 logarithm.

Example:

```
import numpy as np

A = np.array([1, np.e, np.e**2])
print("A:", A)
print("log(A):", np.log(A))
print("exp(A):", np.exp([0, 1, 2])) # exp(0)=1, exp(1)=e, exp(2)=e^2
```

```
A: [1.          2.71828183  7.3890561 ]
log(A): [0.  1.  2.]
exp(A): [1.          2.71828183  7.3890561 ]
```

12.4 Rounding functions and absolute values

- `np.round` — rounds to the nearest number,
- `np.floor` — floor,
- `np.ceil` — ceiling,
- `np.trunc` — truncation to the integer part,
- `np.abs` — absolute value.

Example:

```
import numpy as np

B = np.array([1.7, -2.5, 3.5, 4.52, -4.1])
print("B:", B)
print("floor(B):", np.floor(B))
print("ceil(B):", np.ceil(B))
print("abs(B):", np.abs(B))
print("round(B):", np.round(B))
print("rint(B):", np.rint(B))
print("round(B,1):", np.round(B,1))
```

```
B: [ 1.7 -2.5  3.5  4.52 -4.1 ]
floor(B): [ 1. -3.  3.  4. -5.]
ceil(B): [ 2. -2.  4.  5. -4.]
abs(B): [1.7  2.5  3.5  4.52 4.1 ]
round(B): [ 2. -2.  4.  5. -4.]
rint(B): [ 2. -2.  4.  5. -4.]
round(B,1): [ 1.7 -2.5  3.5  4.5 -4.1]
```

12.5 Statistical and aggregate functions

Although many statistical functions are available as array methods (e.g. `A.mean()`, `A.std()`), there are also ufuncs that operate element-by-element or accept axis parameters:

- `np.minimum`, `np.maximum` — return the element-by-element minimum/maximum of two arrays,
- `np.fmin`, `np.fmax` — similar to the above, but ignore NaN values,
- `np.sqrt` — square root,
- `np.square` — squaring.

Example:

```
import numpy as np

C1 = np.array([1, 4, 9, 16])
C2 = np.array([2, 2, 5, 20])

print("element-wise minimum of C1 and C2:", np.minimum(C1, C2))
print("element-wise maximum of C1 and C2:", np.maximum(C1, C2))
print("sqrt(C1):", np.sqrt(C1))
print("square(C2):", np.square(C2))
```

```
element-wise minimum of C1 and C2: [ 1  2  5 16]
element-wise maximum of C1 and C2: [ 2  4  9 20]
sqrt(C1): [1.  2.  3.  4.]
square(C2): [ 4  4 25 400]
```

13 String operations

In NumPy, besides the well-known numerical arrays, there is also a set of functions that allow vectorized operations on strings.

Important: The functions below are typically available in the `numpy.char` module. In the documentation, they can be found in the [String operations](#) section, but in this material we will focus on how to use them via the `numpy.strings` module interface. This is analogous to using `numpy.char`, but is the newer approach.

13.1 Creating arrays with strings

NumPy allows storing text in arrays, for example:

```
import numpy as np

arr = np.array(["python", "NumPy", "data", "Science"])
print(arr)
```

```
['python' 'NumPy' 'data' 'Science']
```

13.2 Basic text modification functions

Below are popular functions for modifying text in string arrays:

13.2.1 `numpy.strings.upper` and `numpy.strings.lower`

- `upper`: Converts all letters to uppercase.
- `lower`: Converts all letters to lowercase.

```
import numpy as np

arr = np.array(["python", "NumPy", "data", "Science"])

print(np.strings.upper(arr))
print(np.strings.lower(arr))
```

```
['PYTHON' 'NUMPY' 'DATA' 'SCIENCE']
['python' 'numpy' 'data' 'science']
```

13.2.2 `numpy.strings.capitalize`

The `capitalize` function converts the first letter of the entire string to uppercase and the rest to lowercase.

```
import numpy as np

arr = np.array(["python", "NumPy", "data", "Science"])
print(np.strings.capitalize(arr))
```

```
['Python' 'Numpy' 'Data' 'Science']
```

13.2.3 `numpy.strings.title`

The `title` function converts the first letter of each word to uppercase and the remaining letters of each word to lowercase.

```
import numpy as np

arr2 = np.array(["python data science", "machine learning", "deep learning"])
print(np.strings.title(arr2))
```

```
['Python Data Science' 'Machine Learning' 'Deep Learning']
```

13.3 Joining and splitting text

13.3.1 `numpy.strings.add`

The `add` function concatenates elements of text arrays pairwise, acting like the `+` operator on strings, but in a vectorized way. The concatenation is performed without any separator.

```
import numpy as np

arr_a = np.array(["Hello", "Data"])
arr_b = np.array(["World", "Science"])

# without separator
print(np.strings.add(arr_a, arr_b))
# with space -- it must be added manually
print(np.strings.add(np.strings.add(arr_a, np.array([" ", " "]])), arr_b))
```

```
['HelloWorld' 'DataScience']
['Hello World' 'Data Science']
```

13.3.2 `numpy.char.join`

The `join` function inserts a specified separator between individual characters within each element of the array.

```
import numpy as np

arr3 = np.array(["python", "numpy", "string"])
print(np.char.join("-", arr3))
```

```
['p-y-t-h-o-n' 'n-u-m-p-y' 's-t-r-i-n-g']
```

Note: `join` vectorizes the operation, treating each element of the array as a sequence of characters to be joined with the separator.

13.3.3 `numpy.char.split`

Allows splitting strings by a given separator. Returns an array containing lists of substrings.


```
import numpy as np
```

```
arr4 = np.array(["python-data-science", "machine-learning"])  
print(np.char.split(arr4, sep="-"))
```

```
[list(['python', 'data', 'science']) list(['machine', 'learning'])]
```

13.4 Searching and replacing substrings

13.4.1 `numpy.strings.find` and `numpy.strings.rfind`

- `find`: Returns the index of the first occurrence of a substring (or -1 if not found).
- `rfind`: Returns the index of the last occurrence of a substring (or -1 if not found).

```
import numpy as np
```

```
arr5 = np.array(["banana", "data", "abracadabra"])  
# find -- first occurrence  
print(np.strings.find(arr5, "a"))  
# rfind -- last occurrence  
print(np.strings.rfind(arr5, "a"))
```

```
[1 1 0]  
[ 5  3 10]
```

13.4.2 `numpy.strings.replace`

`replace` substitutes all occurrences of a substring with a new string.

```
import numpy as np
```

```
arr6 = np.array(["python", "pydata", "pypy"])  
print(np.strings.replace(arr6, "py", "PY"))
```

```
['PYthon' 'PYdata' 'PYPY']
```

13.5 Removing unwanted characters

13.5.1 `numpy.strings.strip`, `numpy.strings.lstrip` and `numpy.strings.rstrip`

- `strip`: Removes specified characters from the beginning and end.
- `lstrip`: Removes specified characters from the left side (beginning).
- `rstrip`: Removes specified characters from the right side (end).

```
import numpy as np

arr7 = np.array(["  python  ", "  numpy  "])
# strip -- removes from both sides
print(np.strings.strip(arr7))
# lstrip -- removes only from the left
print(np.strings.lstrip(arr7))
# rstrip -- removes only from the right
print(np.strings.rstrip(arr7))
```

```
['python' 'numpy']
['python' 'numpy ']
['  python' '  numpy']
```

We can also specify custom characters to remove:

```
import numpy as np

arr8 = np.array(["###data###", "***science***"])
print(np.strings.strip(arr8, "#*"))
```

```
['data' 'science']
```

14 Linear Algebra

14.1 Dot Product

For two vectors, `dot` computes their dot product.

```
import numpy as np

# Dot product of two vectors
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
result = np.dot(a, b) # 1*4 + 2*5 + 3*6
print(result) # Result: 32

# Alternative notation using the @ operator
result = a @ b
print(result) # Result: 32
```

32

32

14.2 Matrix Multiplication

For matrices (two-dimensional arrays), `dot` performs standard matrix multiplication.

```
import numpy as np
# Matrix multiplication
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
C = np.dot(A, B)
print(C)
# Result:
# [[19 22]
#  [43 50]]
```

```
# The same using the @ operator
C = A @ B
print(C)
```

```
[[19 22]
 [43 50]]
[[19 22]
 [43 50]]
```

14.3 Matrix-Vector Multiplication

We can also multiply a matrix by a vector:

```
import numpy as np
# Matrix-vector multiplication
A = np.array([[1, 2], [3, 4]])
v = np.array([5, 6])
result = np.dot(A, v)
print(result) # Result: [17 39]
```

```
[17 39]
```

14.4 Solving Systems of Linear Equations

The `numpy.linalg.solve` function solves systems of linear equations of the form $Ax = b$:

```
import numpy as np
# Solving a system of linear equations
A = np.array([[3, 1], [1, 2]])
b = np.array([9, 8])
x = np.linalg.solve(A, b)
print(x) # Result: [2. 3.]

# Verifying the solution
np.dot(A, x) # Should be equal to b
```

```
[2. 3.]
```

```
array([9., 8.])
```

14.5 Matrix Determinant

The `numpy.linalg.det` function computes the determinant of a matrix:

```
import numpy as np
# Computing the determinant
A = np.array([[1, 2], [3, 4]])
det_A = np.linalg.det(A)
print(det_A) # Result: -2.0
```

-2.0000000000000004

14.6 Eigenvalues and Eigenvectors

The `numpy.linalg.eig` function computes the eigenvalues and eigenvectors of a matrix:

```
import numpy as np
# Computing eigenvalues and eigenvectors
A = np.array([[4, -2], [1, 1]])
eigenvalues, eigenvectors = np.linalg.eig(A)
print("Eigenvalues:", eigenvalues)
print("Eigenvectors:")
print(eigenvectors)

# Verification: A * v = lambda * v
for i in range(len(eigenvalues)):
    lambda_i = eigenvalues[i]
    v_i = eigenvectors[:, i]
    print(f"_{{i}} = {{lambda_i}}")
    print("A * v =", np.dot(A, v_i))
    print(" * v =", lambda_i * v_i)
```

Eigenvalues: [3. 2.]

Eigenvectors:

[[0.89442719 0.70710678]

[0.4472136 0.70710678]]

_0 = 3.0

A * v = [2.68328157 1.34164079]

* v = [2.68328157 1.34164079]

_1 = 2.0

```
A * v = [1.41421356 1.41421356]
* v = [1.41421356 1.41421356]
```

14.7 Singular Value Decomposition (SVD)

SVD is one of the key methods used in data analysis:

```
import numpy as np
# SVD decomposition
A = np.array([[1, 2], [3, 4], [5, 6]])
U, s, Vh = np.linalg.svd(A)
print("Matrix U:")
print(U)
print("Singular values:", s)
print("Matrix V^H:")
print(Vh)

# Reconstructing matrix A
S = np.zeros((A.shape[0], A.shape[1]))
S[:len(s), :len(s)] = np.diag(s)
A_reconstructed = U @ S @ Vh
print("Reconstructed matrix A:")
print(A_reconstructed)
```

```
Matrix U:
[[-0.2298477  0.88346102  0.40824829]
 [-0.52474482  0.24078249 -0.81649658]
 [-0.81964194 -0.40189603  0.40824829]]
Singular values: [9.52551809 0.51430058]
Matrix V^H:
[[-0.61962948 -0.78489445]
 [-0.78489445  0.61962948]]
Reconstructed matrix A:
[[1. 2.]
 [3. 4.]
 [5. 6.]]
```

14.8 Matrix/Vector Norm

NumPy offers various types of vector and matrix norms:

```

import numpy as np

v = np.array([3, 4])
# L1 (sum of absolute values): |3| + |4| = 7
print("L1 norm:", np.linalg.norm(v, 1))
# L2 / Euclidean (square root of sum of squares): sqrt(9+16) = 5
print("L2 norm (Euclidean):", np.linalg.norm(v))
# Maximum norm (largest absolute value): max(3,4) = 4
print("Maximum norm:", np.linalg.norm(v, np.inf))

A = np.array([[1, 2], [3, 4]])
# Frobenius (like L2, but for matrices): sqrt(1+4+9+16) = 5.477
print("Frobenius norm:", np.linalg.norm(A, 'fro'))

```

```

L1 norm: 7.0
L2 norm (Euclidean): 5.0
Maximum norm: 4.0
Frobenius norm: 5.477225575051661

```

14.9 Inverse Matrix

The `numpy.linalg.inv` function computes the inverse of a matrix:

```

import numpy as np
# Inverse matrix
A = np.array([[1, 2], [3, 4]])
A_inv = np.linalg.inv(A)
print("Inverse matrix:")
print(A_inv)

# Verification: A * A^(-1) = I
print("A * A^(-1):")
print(np.dot(A, A_inv)) # Should be close to the identity matrix

```

```

Inverse matrix:
[[-2.   1.]
 [ 1.5 -0.5]]
A * A^(-1):
[[1.0000000e+00 0.0000000e+00]
 [8.8817842e-16 1.0000000e+00]]

```

14.10 The `numpy.inner` Function - Inner Product

The `inner` function computes the inner product of two arrays:

```
import numpy as np
# Inner product
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
result = np.inner(a, b)
print(result)  # 1*4 + 2*5 + 3*6 = 32

# For 2D arrays: inner(A, B) computes A @ B.T (not A @ B!)
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
result = np.inner(A, B)
print(result)
# [[1*5+2*6, 1*7+2*8],    = [[17, 23],
#   [3*5+4*6, 3*7+4*8]]    [39, 53]]
print("Comparison with A @ B.T:")
print(A @ B.T)
```

```
32
[[17 23]
 [39 53]]
Comparison with A @ B.T:
[[17 23]
 [39 53]]
```

14.11 The `numpy.outer` Function - Outer Product

The `outer` function computes the outer product of two vectors:

```
import numpy as np
# Outer product
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
result = np.outer(a, b)
print(result)
# Result:
# [[ 4  5  6]
```



```
# [ 8 10 12]
# [12 15 18]]
```

```
[[ 4  5  6]
 [ 8 10 12]
 [12 15 18]]
```

14.12 The `numpy.matmul` Function - Matrix Multiplication

The `matmul` function (and the equivalent `@` operator) is the recommended way to multiply matrices. For 2D arrays, it works identically to `dot`. Differences appear for 3D+ arrays (`matmul` treats them as stacks of matrices) and for scalars (`matmul` does not accept scalars, `dot` does).

```
import numpy as np

# For 2D -- identical results
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

print("dot:", np.dot(a, b)[0])
print("matmul:", np.matmul(a, b)[0])

# Difference: matmul does not accept scalars
try:
    np.matmul(a, 3)
except ValueError as e:
    print("matmul with scalar:", e)

print("dot with scalar:", np.dot(a, 3))
```

```
dot: [19 22]
```

```
matmul: [19 22]
```

```
matmul with scalar: matmul: Input operand 1 does not have enough dimensions (has 0, gufunc c
```

```
dot with scalar: [[ 3  6]
```

```
 [ 9 12]]
```

15 Advanced Filtering

15.1 The `nonzero()` Function

Returns the indices of non-zero elements in an array. The result is returned as a tuple of arrays, one for each dimension of the array.

```
import numpy as np

arr = np.array([[3, 0, 0], [0, 4, 0], [5, 6, 0]])
indices = np.nonzero(arr)
print(indices)  # (array([0, 1, 2, 2]), array([0, 1, 0, 1]))

# Extracting non-zero values
values = arr[indices]
print(values)  # [3 4 5 6]

# Alternatively, you can use:
position_indices = np.argwhere(arr != 0)
print(position_indices)
# [[0 0]
#   [1 1]
#   [2 0]
#   [2 1]]
```

```
(array([0, 1, 2, 2]), array([0, 1, 0, 1]))
[3 4 5 6]
[[0 0]
 [1 1]
 [2 0]
 [2 1]]
```

The `np.nonzero()` function returns the indices of all non-zero elements in an array. The result is a tuple of arrays, one per dimension: the first array contains the row indices and the second contains the column indices. For example, given `arr = np.array([[3, 0, 0], [0, 4, 0],`

[5, 6, 0]]), calling `np.nonzero(arr)` returns (`array([0, 1, 2, 2])`, `array([0, 1, 0, 1])`). This means the non-zero elements are located at positions (0,0), (1,1), (2,0), and (2,1), corresponding to the values 3, 4, 5, and 6 respectively.

15.2 The `where()` Function

Returns elements chosen from `x` or `y` depending on a condition. It works like a conditional element selection (equivalent to a ternary expression).

```
import numpy as np

# Replace non-positive values with 0 (zero effectively unchanged)
arr = np.array([1, -2, 3, -4, 5])
result = np.where(arr > 0, arr, 0)
print(result)  # [1 0 3 0 5]

# In a 2D array -- replace negative values with -1
arr_2d = np.array([[1, -2, 3], [-4, 5, -6]])
result_2d = np.where(arr_2d < 0, -1, arr_2d)
print(result_2d)
# [[ 1 -1  3]
#  [-1  5 -1]]
```

```
[1 0 3 0 5]
[[ 1 -1  3]
 [-1  5 -1]]
```

The `np.where(condition, x, y)` function takes three arguments: a condition, a value to use when the condition is true, and a value to use when the condition is false. For each element, it evaluates the condition and returns the corresponding value from `x` if the result is true, or from `y` if the result is false. It acts as an element-wise ternary operator for NumPy arrays.

15.3 The `indices()` and `ix_()` Functions

15.3.1 `indices()`

Creates an array representing grid indices.

```
import numpy as np

# Creating a 3x4 index grid
grid = np.indices((3, 4))
print(grid.shape)  # (2, 3, 4)
print(grid[0])  # row indices
# [[0 0 0 0]
#   [1 1 1 1]
#   [2 2 2 2]]
print(grid[1])  # column indices
# [[0 1 2 3]
#   [0 1 2 3]
#   [0 1 2 3]]
```

```
(2, 3, 4)
[[0 0 0 0]
 [1 1 1 1]
 [2 2 2 2]]
[[0 1 2 3]
 [0 1 2 3]
 [0 1 2 3]]
```

The `np.indices()` function generates an array representing the index grid for a given shape. For a 3×4 grid, `np.indices((3, 4))` returns a result of shape (2, 3, 4), where the first component (`grid[0]`) contains the row indices and the second component (`grid[1]`) contains the column indices. In `grid[0]`, each row is filled with its own row number (all zeros in the first row, all ones in the second, all twos in the third), while in `grid[1]`, each row contains the sequence of column numbers 0, 1, 2, 3 repeated identically across all rows. Together, these two arrays allow you to reference every position in the grid by its (row, column) pair.

15.3.2 `ix_()`

Constructs an open mesh from multiple sequences, which is useful for multidimensional indexing.

```
import numpy as np
x = np.array([0, 1, 2])
y = np.array([3, 4, 5, 6])
indices = np.ix_(x, y)

# Creates an open index grid for broadcasting
```

```

print(indices[0].shape, indices[1].shape) # (3, 1) (1, 4)

# Using it to select a subarray
arr = np.arange(16).reshape(4, 4)
print(arr)
# [[ 0  1  2  3]
#   [ 4  5  6  7]
#   [ 8  9 10 11]
#   [12 13 14 15]]

subarray = arr[np.ix_([0, 2, 3], [0, 2])]
print(subarray)
# [[ 0  2]
#   [ 8 10]
#   [12 14]]

```

```

(3, 1) (1, 4)
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]
[[ 0  2]
 [ 8 10]
 [12 14]]

```

15.4 ogridd and Grid Operations

ogrid allows creating open grids, which is more memory-efficient than full grids.

```

import numpy as np

# Grid of points in the range from -2 to 2 with step 0.1
x, y = np.ogrid[-2:2:0.1, -2:2:0.1]
mask = x**2 + y**2 <= 1 # Circle with radius 1
print(mask.shape) # (40, 40)

```

```

(40, 40)

```

15.5 The `ravel_multi_index()` and `unravel_index()` Functions

These functions convert between multidimensional and flat indices.

```
import numpy as np

# Converting multidimensional indices to flat indices
multi_indices = np.array([[0, 0], [1, 1], [2, 1]])
dimensions = (3, 3)
flat_indices = np.ravel_multi_index(multi_indices.T, dimensions)
print(flat_indices)  # [0 4 7]

# Converting flat indices to multidimensional indices
flat_indices = np.array([0, 3, 8])
shape = (3, 3)
multi_indices = np.unravel_index(flat_indices, shape)
print(multi_indices)  # (array([0, 1, 2]), array([0, 0, 2]))
```

```
[0 4 7]
(array([0, 1, 2]), array([0, 0, 2]))
```

15.6 Diagonal Indices

NumPy offers many functions for working with matrix diagonals.

```
import numpy as np

# Getting the main diagonal indices
n = 4
diag_indices = np.diag_indices(n)
print(diag_indices)  # (array([0, 1, 2, 3]), array([0, 1, 2, 3]))

# Using them to set the main diagonal
arr = np.zeros((4, 4))
arr[diag_indices] = 1  # Setting ones on the main diagonal
print(arr)
# [[1. 0. 0. 0.]
#  [0. 1. 0. 0.]
#  [0. 0. 1. 0.]
#  [0. 0. 0. 1.]]
```

```
# Getting indices from an existing array
arr2 = np.ones((3, 3))
diag_indices2 = np.diag_indices_from(arr2)
print(diag_indices2)
```

```
(array([0, 1, 2, 3]), array([0, 1, 2, 3]))
[[1.  0.  0.  0.]
 [0.  1.  0.  0.]
 [0.  0.  1.  0.]
 [0.  0.  0.  1.]]
(array([0, 1, 2]), array([0, 1, 2]))
```

15.7 The take() Function

Retrieves elements from an array along a specified axis based on indices.

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])
indices = np.array([0, 2, 4])
result = np.take(arr, indices)
print(result) # [10 30 50]

# In multidimensional arrays, we can choose the axis
arr_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
row_indices = np.array([0, 2])
result_2d = np.take(arr_2d, row_indices, axis=0)
print(result_2d)
# [[1 2 3]
#    [7 8 9]]
```

```
[10 30 50]
[[1 2 3]
 [7 8 9]]
```

15.8 The take_along_axis() Function

Retrieves values from an array along a specified axis, using an index array with a compatible shape. Useful, for example, for applying results from `argsort()` or `argmin()`.

```
import numpy as np

arr = np.array([[10, 30, 20], [60, 40, 50]])
sort_indices = np.argsort(arr, axis=1)
result = np.take_along_axis(arr, sort_indices, axis=1)
print(result)
# [[10 20 30]
#   [40 50 60]]
```

```
[[10 20 30]
 [40 50 60]]
```

15.9 The choose() Function

Constructs an array by selecting elements from a list of arrays.

```
import numpy as np

choices = [np.array([0, 1, 2, 3]),
            np.array([10, 11, 12, 13]),
            np.array([20, 21, 22, 23])]
indices = np.array([0, 2, 1, 0]) # Selects which choice array to take the element from
result = np.choose(indices, choices)
print(result) # [ 0 21 12  3]
```

```
[ 0 21 12  3]
```

15.10 The compress() Function

Returns selected elements of an array along a specified axis.

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
mask = np.array([True, False, True])
result = np.compress(mask, arr, axis=1)
print(result)
# [[1 3]
#   [4 6]]
```



```
[[1 3]
 [4 6]]
```

15.11 The `diag()` and `diagonal()` Functions

Functions for working with diagonals.

```
import numpy as np

# Creating a diagonal array
diag_arr = np.diag([1, 2, 3, 4])
print(diag_arr)
# [[1 0 0 0]
#   [0 2 0 0]
#   [0 0 3 0]
#   [0 0 0 4]]

# Extracting the diagonal from an array
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
diag = np.diag(arr)
print(diag) # [1 5 9]

# Extracting a diagonal shifted by 1
diag_offset = np.diagonal(arr, offset=1)
print(diag_offset) # [2 6]
```

```
[[1 0 0 0]
 [0 2 0 0]
 [0 0 3 0]
 [0 0 0 4]]
[1 5 9]
[2 6]
```

15.12 The `select()` Function

Returns an array built from elements of a list of choices, depending on conditions.

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
conditions = [arr < 3, arr < 6, arr < 9]
choices = [100, 200, 300]
result = np.select(conditions, choices, default=400)
print(result)
# [[100 100 200]
#   [200 200 300]
#   [300 300 400]]
```

```
[[100 100 200]
 [200 200 300]
 [300 300 400]]
```

15.13 The place() Function

Changes elements of an array based on a mask and provided values.

```
import numpy as np

arr = np.arange(5)
mask = np.array([True, False, True, False, True])
np.place(arr, mask, [-1, -2, -3]) # Cyclically uses values [-1, -2, -3]
print(arr) # [-1  1 -2  3 -3]
```

```
[-1  1 -2  3 -3]
```

15.14 The put() Function

Replaces specified elements of an array with given values.

```
import numpy as np

arr = np.arange(5)
indices = [0, 2, 4]
np.put(arr, indices, [10, 20, 30])
print(arr) # [10  1 20  3 30]
```

```
[10  1 20  3 30]
```

15.15 The `put_along_axis()` Function

Inserts values into a target array along a specified axis, using an index array with a compatible shape.

```
import numpy as np

arr = np.array([[10, 30, 20], [60, 40, 50]])
indices = np.argmax(arr, axis=1)
indices = np.expand_dims(indices, axis=1) # Reshape to shape (2, 1)
np.put_along_axis(arr, indices, 99, axis=1)
print(arr)
# [[99 30 20]
#   [60 99 50]]
```

```
[[99 30 20]
 [60 99 50]]
```

15.16 The `putmask()` Function

Changes elements of an array based on a mask. Unlike `place()`, values are assigned positionally (with cyclic extension to the array size), not sequentially to elements satisfying the mask.

```
import numpy as np

arr = np.arange(5)
mask = np.array([True, False, True, False, True])
np.putmask(arr, mask, [-1, -2, -3]) # Values assigned positionally (cyclically)
print(arr) # [-1  1 -3  3 -2]
```

```
[-1  1 -3  3 -2]
```

15.17 The `fill_diagonal()` Function

Fills the main diagonal of an array with a given value.

```

import numpy as np

arr = np.zeros((4, 4))
np.fill_diagonal(arr, 5)
print(arr)
# [[5. 0. 0. 0.]
#  [0. 5. 0. 0.]
#  [0. 0. 5. 0.]
#  [0. 0. 0. 5.]]

arr_rect = np.zeros((4, 4, 4))
np.fill_diagonal(arr_rect, 9)
print(arr_rect[0]) # Only element [0][0] set to 9 (super-diagonal i,i,i)

```

```

[[5. 0. 0. 0.]
 [0. 5. 0. 0.]
 [0. 0. 5. 0.]
 [0. 0. 0. 5.]]
[[9. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]

```

16 NumPy - Miscellaneous

16.1 Constants

NumPy provides several well-known mathematical constants that can be useful in scientific and engineering computations. Built-in constants such as the number Pi or the base of the natural logarithm `e` make it easier to write readable and concise code.

1. `numpy.pi`

- Represents the number Pi (π) with high precision.
- Pi is the ratio of a circle's circumference to its diameter.
- Approximately: 3.141592653589793

2. `numpy.e`

- Represents the base of the natural logarithm, `e`.
- `e` is used in many fields, such as mathematical analysis, probability theory, and statistics.
- Approximately: 2.718281828459045

3. `numpy.eulergamma`

- Represents the Euler-Mascheroni constant, usually denoted as γ (gamma).
- It appears in mathematical analysis, particularly in number theory and the study of harmonic series.
- Approximately: 0.5772156649015329

```
import numpy as np

# Circle radius
r = 5.0

# Circle circumference: 2 * pi * r
circumference = 2 * np.pi * r
```

```
print("Circle circumference:", circumference)

# Circle area:    * r^2
area = np.pi * r**2
print("Circle area:", area)
```

```
Circle circumference: 31.41592653589793
Circle area: 78.53981633974483
```

```
import numpy as np

# Sample point x
x = 1.0

# Value of the function e^x
exp_value = np.e**x
print("e^x for x=1:", exp_value)

# Comparison with the np.exp function
exp_compare = np.exp(x)
print("Comparison with np.exp(1):", exp_compare)
```

```
e^x for x=1: 2.718281828459045
Comparison with np.exp(1): 2.718281828459045
```

16.2 numpy.inf

- **Description:** `np.inf` represents an infinite value (∞).
- It often appears in computations when the value of an expression tends to infinity (e.g., division by zero, certain limits, etc.).
- For example, `1.0 / 0.0` will return a warning and may consequently yield the value `inf`.

```
import numpy as np

# Using it to create boolean masks
arr = np.array([1, 2, np.inf, 4, 5])
mask = np.isinf(arr)
print("Mask of elements with value inf:", mask)
```

Mask of elements with value inf: [False False True False False]

16.3 numpy.nan

- **Description:** `np.nan` stands for “Not a Number” (NaN), meaning an undefined or unrepresentable value in the number system.
- It appears when the result of a numerical operation is undefined, e.g., `0.0/0.0`, `inf - inf`, or when data loading errors occur.
- Arithmetic operations with `nan` typically also return `nan`.

```
import numpy as np

# Replacing nan values in an array
data = np.array([1, 2, np.nan, 4, np.nan])
print("Original data:", data)

# Filling nan values with zero
data_no_nan = np.nan_to_num(data, nan=0.0)
print("Data without nan:", data_no_nan)
```

Original data: [1. 2. nan 4. nan]
Data without nan: [1. 2. 0. 4. 0.]

16.4 numpy.newaxis

- **Description:** `np.newaxis` is a special “constant”/object used to change array dimensions by increasing the number of dimensions by 1.

```
import numpy as np

# We have a 1D array
vec = np.array([1, 2, 3, 4])
print("Original array:", vec, "Shape:", vec.shape)

# Adding a new dimension as the row dimension
vec_as_col = vec[:, np.newaxis]
print("Array as column:\n", vec_as_col, "Shape:", vec_as_col.shape)
```

```
# Adding a dimension at the beginning
vec_as_row = vec[np.newaxis, :]
print("Array as row:\n", vec_as_row, "Shape:", vec_as_row.shape)

# Another example: adding a dimension to easily broadcast operations
a = np.array([10, 20, 30])
b = np.array([1, 2])
# Without a new dimension, trying to add a to b would fail,
# because the shapes are not compatible.
# With a new dimension, a has shape (3,1) and b (2,), which allows broadcasting
sum_matrix = a[:, np.newaxis] + b
print("Operation with broadcasting:\n", sum_matrix)
```

Original array: [1 2 3 4] Shape: (4,)

Array as column:

```
[[1]
 [2]
 [3]
 [4]] Shape: (4, 1)
```

Array as row:

```
[[1 2 3 4]] Shape: (1, 4)
```

Operation with broadcasting:

```
[[11 12]
 [21 22]
 [31 32]]
```

16.5 Statistics and Aggregation

Function	Description
np.mean	Mean of all values in the array.
np.std	Standard deviation.
np.var	Variance.
np.sum	Sum of all elements.
np.prod	Product of all elements.
np.cumsum	Cumulative sum of all elements.
np.cumprod	Cumulative product of all elements.
np.min, np.max	Minimum/maximum value in the array.
np.argmin, np.argmax	Index of the minimum/maximum value in the array.
np.all	Checks whether all elements are non-zero.

Function	Description
<code>np.any</code>	Checks whether at least one element is non-zero.